

Relatorio tecnico - Algoritmos geneticos

RELATORIO: OTIMIZACAO DO MODELO TABULAR COM ALGORITMO GENETICO

1. OBJETIVO

O problema consiste em classificar se um registro deve ser considerado perigoso (`Dangerous = Yes`) ou nao perigoso (`Dangerous = No`) a partir do animal e de cinco sintomas.

O experimento foi planejado desde o inicio para otimizar geneticamente quatro modelos, cada um com seu proprio espaco de hiperparametros:

- Regressao Logistica;
- Arvore de Decisao;
- Random Forest;
- KNN.

O objetivo foi encontrar, para cada modelo, uma combinacao de hiperparametros que maximizasse um fitness composto por `accuracy`, `recall` e `F1`. Foi usada a formula $0.30 * accuracy + 0.40 * recall + 0.30 * F1$, dando maior peso ao recall da classe perigosa. Em um cenario de triagem, deixar de identificar um caso perigoso pode ser mais grave do que gerar um alerta falso.

2. ESTADO ANTES DA OTIMIZACAO

O pipeline original executava as seguintes etapas:

1. Carregava e normalizava o dataset.
2. Removia duplicatas e preenchia sintomas ausentes com `unknown`.
3. Criava as features `unique\_symptom\_count`, `unknown\_symptom\_count` e `symptom\_text\_length`.
4. Codificava as variaveis categoricas com `OneHotEncoder`.
5. Padronizava as variaveis numericas com `StandardScaler`.
6. Separava os dados em treino, validacao e teste, preservando a proporcao das classes.
7. Treinava quatro modelos.
8. Escolhia o melhor modelo pela ordem `recall\_yes`, `f1\_yes` e `accuracy` na validacao.

Como referencia, o Random Forest original usava:

```
RandomForestClassifier(  
    n_estimators=350,  
    min_samples_leaf=2,  
    class_weight="balanced",  
    random_state=random_state,  
    n_jobs=-1,  
)
```

Relatorio tecnico - Algoritmos geneticos

O resultado do melhor modelo original, que foi o KNN, foi:

Conjunto	Modelo	Accuracy	Recall	Yes	F1	Yes	ROC	AUC
---	---	---	---	---	---	---	---	---
Validacao	KNN	0.9762	1.0000	0.9880	0.8369			
Teste	KNN	0.9763	1.0000	0.9880	0.9833			

3. PREPARACAO DA BUSCA GENETICA PARA OS QUATRO MODELOS

A mesma estrutura genetica foi aplicada aos quatro modelos. O que muda entre eles e o espaco de hiperparametros; a avaliacao, a validacao cruzada, a selecao, o cruzamento, a mutacao e o criterio de comparacao permanecem os mesmos.

Foram importados os recursos necessarios:

```
from copy import deepcopy
from sklearn.metrics import make_scorer
from sklearn.model_selection import StratifiedKFold, cross_validate
```

`deepcopy` preserva uma copia independente do melhor individuo. `StratifiedKFold` cria as divisoes da validacao cruzada mantendo a proporcao entre as classes. `cross\_validate` calcula accuracy, recall e F1 para formar o fitness.

A funcao principal recebeu os dados de treino e as colunas do pipeline:

```
def genetic_search_model(
    split: SplitData,
    categorical_cols: List[str],
    numeric_cols: List[str],
    random_state: int,
    population_size: int = 12,
    generations: int = 8,
) -> Tuple[Pipeline, dict, pd.DataFrame]:
```

Foram usados 12 individuos por geracao e 8 geracoes. Esses valores mantem o custo computacional razoavel para o dataset do projeto.

4. FUNCAO OBJETIVO

A busca foi configurada para maximizar o recall da classe positiva:

```
rng = np.random.default_rng(random_state)
```

Relatorio tecnico - Algoritmos geneticos

```
cv = StratifiedKfold(n_splits=5, shuffle=True, random_state=random_state)
recall_scorer = make_scorer(
    recall_score,
    pos_label=POSITIVE_LABEL,
    zero_division=0,
)
```

0 `random\_state` torna a busca reproduzível. A validacao cruzada possui cinco partes. Em cada rodada, quatro partes sao usadas para treino e uma para validacao.

5. REPRESENTACAO DOS INDIVIDUOS

Cada individuo representa uma configuracao completa de um dos quatro modelos. A estrutura do individuo muda conforme o modelo e e definida pelo espaco genetico correspondente:

```
def random_individual() -> dict:
    return {
        "n_estimators": int(rng.integers(150, 601)),
        "max_depth": int(rng.integers(4, 26)),
        "min_samples_split": int(rng.integers(2, 16)),
        "min_samples_leaf": int(rng.integers(1, 9)),
        "max_features": str(rng.choice(["sqrt", "log2"])),
        "class_weight": str(rng.choice(["balanced", "balanced_subsample"])),
    }
```

Os genes controlam:

- `n\_estimators`: quantidade de arvores;
- `max\_depth`: profundidade maxima das arvores;
- `min\_samples\_split`: quantidade minima de amostras para dividir um no;
- `min\_samples\_leaf`: quantidade minima de amostras em uma folha;
- `max\_features`: quantidade de features analisadas em cada divisao;
- `class\_weight`: tratamento do peso das classes.

Os intervalos evitam configuracoes extremamente pequenas ou muito custosas.

6. AVALIACAO DE UM INDIVIDUO

Cada configuracao foi avaliada dentro do mesmo pipeline de preprocessamento:

```
def evaluate(individual: dict) -> float:
    estimator = Pipeline(
```

Raportrio tecnico - Algoritmos geneticos

```
[
    ("preprocess", build_preprocessor(categorical_cols, numeric_cols)),
    (
        "model",
        RandomForestClassifier(
            **individual,
            random_state=random_state,
            n_jobs=-1,
        ),
    ),
]
)
scores = cross_val_score(
    estimator,
    split.x_train,
    split.y_train,
    scoring=recall_scorer,
    cv=cv,
    n_jobs=-1,
)
return float(scores.mean())
```

Somente `split.x\_train` e `split.y\_train` participam da busca. O conjunto de teste fica isolado para a avaliacao final, evitando vazamento de informacao.

7. MUTACAO

A mutacao escolhe um gene aleatorio e sorteia um novo valor para ele:

```
def mutate(individual: dict) -> dict:
    child = individual.copy()
    gene = str(rng.choice(list(child)))
    if gene == "n_estimators":
        child[gene] = int(rng.integers(150, 601))
    elif gene == "max_depth":
        child[gene] = int(rng.integers(4, 26))
    elif gene == "min_samples_split":
        child[gene] = int(rng.integers(2, 16))
    elif gene == "min_samples_leaf":
        child[gene] = int(rng.integers(1, 9))
    elif gene == "max_features":
        child[gene] = str(rng.choice(["sqrt", "log2"]))
    else:
        child[gene] = str(rng.choice(["balanced", "balanced_subsample"]))
    return child
```

Relatorio tecnico - Algoritmos geneticos

A mutacao introduz diversidade e reduz a chance de a populacao ficar presa em uma configuracao local.

8. CRUZAMENTO

0 cruzamento combina dois pais gene a gene:

```
def crossover(first: dict, second: dict) -> dict:
    return {
        gene: first[gene] if rng.random() < 0.5 else second[gene]
        for gene in first
    }
```

Cada gene do filho vem aleatoriamente do primeiro ou do segundo pai.

9. SELECAO, ELITISMO E EVOLUCAO

A populacao inicial foi criada aleatoriamente:

```
population = [random_individual() for _ in range(population_size)]
history = []
best_individual = None
best_score = -np.inf
```

A evolucao foi implementada assim:

```
for generation in range(generations):
    scored = [(individual, evaluate(individual)) for individual in population]
    scored.sort(key=lambda item: item[1], reverse=True)
    generation_best, generation_score = scored[0]
    history.append(
        {
            "generation": generation + 1,
            "best_recall_cv": generation_score,
            "mean_recall_cv": float(np.mean([score for _, score in scored])),
        }
    )
    if generation_score > best_score:
        best_individual = deepcopy(generation_best)
        best_score = generation_score
```

Relatorio tecnico - Algoritmos geneticos

```
elite_count = max(2, population_size // 4)
parents = [individual for individual, _ in scored[:elite_count]]
next_population = [individual.copy() for individual in parents]
while len(next_population) < population_size:
    first = parents[int(rng.integers(0, len(parents)))]
    second = parents[int(rng.integers(0, len(parents)))]
    child = crossover(first, second)
    if rng.random() < 0.30:
        child = mutate(child)
    next_population.append(child)
population = next_population
```

A selecao elitista preserva os 25% melhores individuos. Os pais sao escolhidos entre os melhores, os filhos sao criados por cruzamento e 30% dos filhos sofrem mutacao.

10. TREINAMENTO FINAL

Depois das geracoes, o melhor individuo foi usado para criar e treinar o modelo final:

```
if best_individual is None:
    raise RuntimeError("A busca genetica nao encontrou uma configuracao.")

best_model = Pipeline(
    [
        ("preprocess", build_preprocessor(categorical_cols, numeric_cols)),
        (
            "model",
            RandomForestClassifier(
                **best_individual,
                random_state=random_state,
                n_jobs=-1,
            ),
        ),
    ]
)
best_model.fit(split.x_train, split.y_train)
return best_model, best_individual, pd.DataFrame(history)
```

11. AJUSTE DO LIMIAR DE DECISAO

A busca genetica otimiza os hiperparametros, mas o limiar de classificacao tambem influencia o recall e o F1. Por isso, foram testados limiares entre 0.10 e 0.90 na validacao:

Relatorio tecnico - Algoritmos geneticos

```
def find_validation_threshold(model: Pipeline, split: SplitData) -> Tuple[float, pd.DataFrame]:
    probabilities = model.predict_proba(split.x_val)[: , 1]
    threshold_records = []

    for threshold in np.arange(0.10, 0.91, 0.01):
        predictions = (probabilities >= threshold).astype(int)
        threshold_records.append(
            {
                "threshold": round(float(threshold), 2),
                "recall_yes": recall_score(
                    split.y_val,
                    predictions,
                    pos_label=POSITIVE_LABEL,
                    zero_division=0,
                ),
                "f1_yes": f1_score(
                    split.y_val,
                    predictions,
                    pos_label=POSITIVE_LABEL,
                    zero_division=0,
                ),
            }
        )

    table = pd.DataFrame(threshold_records).sort_values(
        ["recall_yes", "f1_yes"],
        ascending=False,
    )
    return float(table.iloc[0]["threshold"]), table
```

O limiar escolhido foi `0.38`. A regra prioriza o maior recall e usa o F1 como desempate.

12. HIPERPARAMETROS ENCONTRADOS

```
{
    "n_estimators": 188,
    "max_depth": 19,
    "min_samples_split": 4,
    "min_samples_leaf": 1,
    "max_features": "log2",
    "class_weight": "balanced_subsample"
}
```

Raportorio tecnico - Algoritmos geneticos

O algoritmo atingiu `recall medio na validacao cruzada = 1.0000` nas oito geracoes.

13. COMPARACAO DE DESEMPENHO DA EXECUCAO INICIAL

Esta tabela registra a execucao inicial documentada no notebook. A tabela oficial e completa, com os quatro modelos antes e depois, esta na secao 17.

Modelo	Conjunto	Accuracy	Precision Yes	Recall Yes	F1 Yes	F1 Weighted	ROC AUC
---	---	---	---	---	---	---	---
KNN original	Validacao	0.9762	0.9762	1.0000	0.9880	0.9644	0.8369
KNN original	Teste	0.9763	0.9763	1.0000	0.9880	0.9646	0.9833
Random Forest genetico	Validacao	0.9821	0.9820	1.0000	0.9909	0.9769	0.7790
Random Forest genetico	Teste	0.9941	0.9940	1.0000	0.9970	0.9937	0.9697

ANALISE

- O modelo otimizado manteve o recall da classe perigosa em 100%.
- A accuracy no teste subiu de 97,63% para 99,41%, uma melhora de 1,78 ponto percentual.
- O F1 da classe perigosa subiu de 0,9880 para 0,9970.
- O F1 weighted subiu de 0,9646 para 0,9937.
- O ROC AUC do Random Forest ficou menor que o ROC AUC do KNN no teste, embora suas classificacoes no limiar escolhido tenham sido melhores. Isso mostra que uma unica metrica nao deve ser usada isoladamente.
- O resultado deve ser interpretado como apoio a triagem, pois o dataset e limitado e possui categorias textuais muito especificas.

14. ARTEFATOS GERADOS

- `data.csv`: dataset utilizado.
- `outputs\_techchallenge\_b/models/best\_model.pkl`: modelo otimizado salvo.
- `outputs\_techchallenge\_b/models/genetic\_best\_params.json`: hiperparametros vencedores.
- `outputs\_techchallenge\_b/models/genetic\_threshold.json`: limiar otimizado.
- `outputs\_techchallenge\_b/tables/genetic\_search\_history.csv`: desempenho por geracao.
- `outputs\_techchallenge\_b/tables/threshold\_results.csv`: avaliacao dos limiares.
- `outputs\_techchallenge\_b/tables/optimized\_model\_metrics.csv`: metricas finais.
- `outputs\_techchallenge\_b/reports/raportorio\_tecnico.md`: raportorio tecnico do pipeline.
- `outputs\_techchallenge\_b/reports/raportorio\_tecnico.pdf`: raportorio em PDF.

15. COMO APRESENTAR O ANTES E DEPOIS

Use o notebook atual como versao depois: ele contem as celulas de download, busca genetica e persistencia do modelo. Para a versao antes, mantenha apenas a celula principal do pipeline e as celulas de visualizacao/teste, removendo as celulas intituladas:

Relatorio tecnico - Algoritmos geneticos

- `Download automatico do dataset tabular antes do pipeline`;
- `Localizar ou baixar automaticamente o dataset`;
- `Otimizacao genetica do Random Forest e ajuste do limiar de decisao`;
- `Persistir o modelo otimizado para as celulas de teste e para a API`.

A comparacao deve mostrar que o antes usa hiperparametros fixos, enquanto o depois evolui configuracoes dos quatro modelos com validacao cruzada e ajusta o limiar de decisao.

16. EXTENSAO: ALGORITMO GENETICO NOS QUATRO MODELOS

A funcao `genetic\_search\_model` recebe o nome de cada modelo e usa um espaco de genes especifico:

Modelo	Genes otimizados
Regressao Logistica	`C`, `penalty`, `class_weight`
Arvore de Decisao	`max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `class_weight`
Random Forest	`n_estimators`, `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`, `class_weight`
KNN	`n_neighbors`, `weights`, `p`, `leaf_size`

O codigo comum chama `build\_model\_from\_genes`, avalia com `cross\_val\_score`, preserva os melhores individuos, faz cruzamento e aplica mutacao. Assim, a logica genetica nao e duplicada quatro vezes.

HIPERPARAMETROS ENCONTRADOS NA EXECUCAO MAIS RECENTE

{	"logistic_regression": {"C": 0.01, "penalty": "l2", "class_weight": null},
	"decision_tree": {"max_depth": 17, "min_samples_split": 20, "min_samples_leaf": 8, "max_features":
null,	"class_weight": null},
	"random_forest": {"n_estimators": 190, "max_depth": 21, "min_samples_split": 11, "min_samples_leaf":
4,	"max_features": "log2", "class_weight": null},
	"knn": {"n_neighbors": 12, "weights": "distance", "p": 1, "leaf_size": 47}
}	

COMPARACAO DOS QUATRO MODELOS OTIMIZADOS

Modelo	Validacao Accuracy	Validacao F1	Teste Accuracy	Teste Recall	Teste F1
Regressao Logistica	0.9762	0.9880	0.9763	1.0000	0.9880
Arvore de Decisao	0.9881	0.9939	0.9704	0.9879	0.9849
Random Forest	0.9762	0.9880	0.9763	1.0000	0.9880
KNN	0.9762	0.9880	0.9763	1.0000	0.9880

Relatorio tecnico - Algoritmos geneticos

Embora a Arvore de Decisao tenha vencido a validacao, seu desempenho no teste foi menor. Isso evidencia o risco de selecionar o modelo somente por uma divisao de validacao. Para uma conclusao mais robusta, recomenda-se repetir a busca com varias sementes, usar validacao cruzada aninhada ou escolher o modelo considerando tambem a estabilidade entre as dobras. O resultado anterior do Random Forest, obtido em uma busca com outro espaco e outra populacao, nao deve ser comparado diretamente sem manter as mesmas condicoes experimentais.

17. COMPARACAO COMPLETA ANTES VERSUS DEPOIS

A tabela definitiva foi gerada pelo notebook em `tables/before\_after\_model\_metrics.csv`. Ela avalia os quatro modelos originais e os quatro modelos depois da busca genetica, tanto na validacao quanto no teste.

Versao	Modelo	Conjunto	Accuracy	Recall Yes	F1 Yes	ROC AUC
Original	Regressao Logistica	Validacao	0.9702	0.9878	0.9848	0.9070
Original	Regressao Logistica	Teste	0.9941	1.0000	0.9970	0.9591
Original	Arvore de Decisao	Validacao	0.9107	0.9146	0.9524	0.8567
Original	Arvore de Decisao	Teste	0.9704	0.9758	0.9847	0.8614
Original	Random Forest	Validacao	0.9821	0.9939	0.9909	0.9268
Original	Random Forest	Teste	0.9822	0.9879	0.9909	0.9697
Original	KNN	Validacao	0.9762	1.0000	0.9880	0.8369
Original	KNN	Teste	0.9763	1.0000	0.9880	0.9833
Genetico	Regressao Logistica	Validacao	0.9762	1.0000	0.9880	0.6235
Genetico	Regressao Logistica	Teste	0.9763	1.0000	0.9880	0.6424
Genetico	Arvore de Decisao	Validacao	0.9881	1.0000	0.9939	0.9535
Genetico	Arvore de Decisao	Teste	0.9704	0.9879	0.9849	0.9758
Genetico	Random Forest	Validacao	0.9762	1.0000	0.9880	0.9223
Genetico	Random Forest	Teste	0.9763	1.0000	0.9880	0.9742
Genetico	KNN	Validacao	0.9762	1.0000	0.9880	0.8232
Genetico	KNN	Teste	0.9763	1.0000	0.9880	0.9742

Portanto, agora o relatorio apresenta a comparacao dos quatro modelos antes e depois. A interpretacao deve destacar que a busca genetica aumentou o desempenho de validacao da Arvore de Decisao, mas o teste mostra que essa melhoria nao se generalizou completamente. O melhor resultado de teste entre os modelos avaliados foi o da Regressao Logistica original, com `accuracy=0.9941`, enquanto o melhor recall no teste foi compartilhado pela Regressao Logistica original, KNN original e os quatro modelos geneticos, conforme a tabela.

18. TRES EXPERIMENTOS GENETICOS EXIGIDOS

Foram executados tres experimentos para cada um dos quatro modelos:

Experimento	Populacao	Geracoes	Taxa de mutacao
E1_pop6_mut10	6	4	0.10
E2_pop8_mut30	8	5	0.30
E3_pop12_mut50	12	6	0.50

Relatorio tecnico - Algoritmos geneticos

O fitness de cada individuo foi calculado por:

```
fitness = (  
    0.30 * accuracy_cv  
    + 0.40 * recall_cv  
    + 0.30 * f1_cv  
)
```

Na execucao final, o melhor experimento por modelo foi:

Modelo	Experimento selecionado	Fitness de validacao	
---	---	---	
Regressao Logistica	E3_pop12_mut50	0.9919	
Arvore de Decisao	E1_pop6_mut10	0.9892	
Random Forest	E2_pop8_mut30	0.9919	
KNN	E3_pop12_mut50	0.9919	

Os resultados completos por experimento e geracao foram salvos em `tables/genetic\_experiment\_results.csv` e `tables/genetic\_experiment\_history.csv`. Os hiperparametros vencedores foram salvos em `models/genetic\_best\_params\_all\_experiments.json`.